

# Module 14 - chapter 5

## 5 - Health Check: EC2 Status Checks

The project files used in this lecture can be found here:

Terraform files:

- Starting code: <https://gitlab.com/twn-devops-bootcamp/latest/14-automation-with-python/terraform/-/tree/feature/starting-code>
- Final code: <https://gitlab.com/twn-devops-bootcamp/latest/14-automation-with-python/terraform>

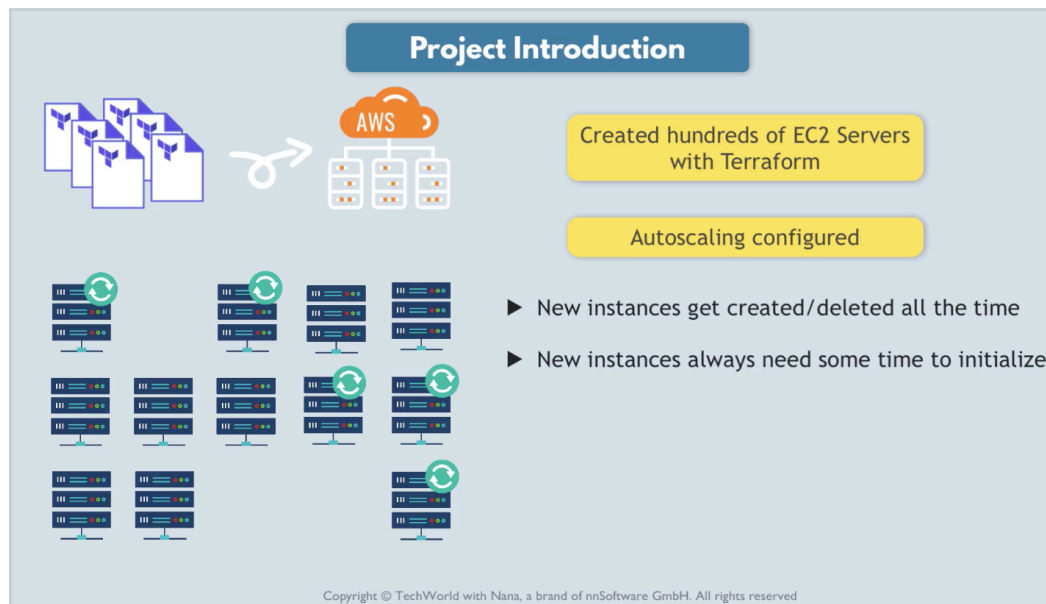
Python code:

- <https://gitlab.com/twn-devops-bootcamp/latest/14-automation-with-python/automation-projects/-/blob/main/ec2-status-checks.py>

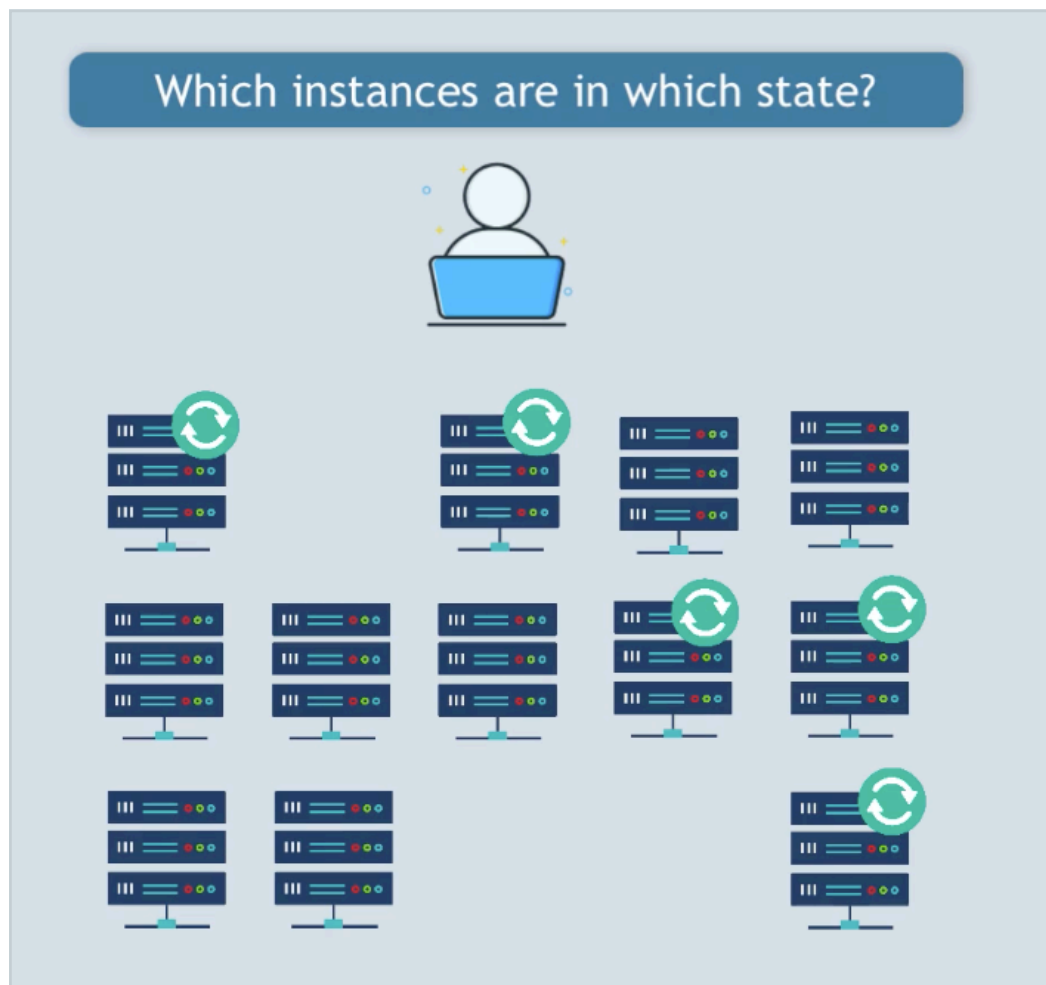
## Project intro

We used TF to deploy 100s of EC2 servers and we have autoscaling configured. so all the time instances are removed and or created.

And remember that EC2 instances need some time to initialise and be up and running



then we want to know which instance is in which state



So we can have an overview

### Project Exercise: EC2 Status Checks

Which instances are in which state?

| <input type="checkbox"/> | Name             | Instance ID         | Instance state | Instance type | Status check |
|--------------------------|------------------|---------------------|----------------|---------------|--------------|
| <input type="checkbox"/> | dev-server-three | i-0092fb0d1ea678c6f | Running        | t2.micro      | Initializing |
| <input type="checkbox"/> | dev-server       | i-0de11486f87f9290a | Running        | t2.micro      | Initializing |
| <input type="checkbox"/> | dev-server-two   | i-0936e85840d5d227f | Running        | t2.micro      | Initializing |

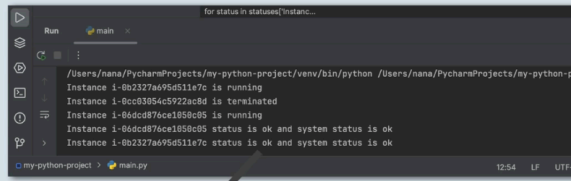
Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

And we can use python boto3 library to get the status check of the servers

## Project Exercise: EC2 Status Checks

Which instances are in which state?



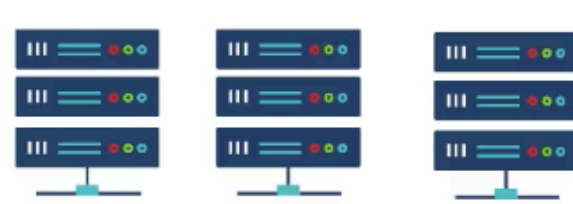


| <input type="checkbox"/> | Name             | Instance ID         | Instance state | Instance type | Status check |
|--------------------------|------------------|---------------------|----------------|---------------|--------------|
| <input type="checkbox"/> | dev-server-three | i-0092fb0d1ea678c6f | Running        | t2.micro      | Initializing |
| <input type="checkbox"/> | dev-server       | i-0de11486f87f9290a | Running        | t2.micro      | Initializing |
| <input type="checkbox"/> | dev-server-two   | i-0936e85840d5d227f | Running        | t2.micro      | Initializing |

For the demo and to keep costs low we will have only 3 servers (logic is the same for 100 or 3 servers)

## Which instances are in which state?





AWS account

First step is to create the EC2 instances using TF

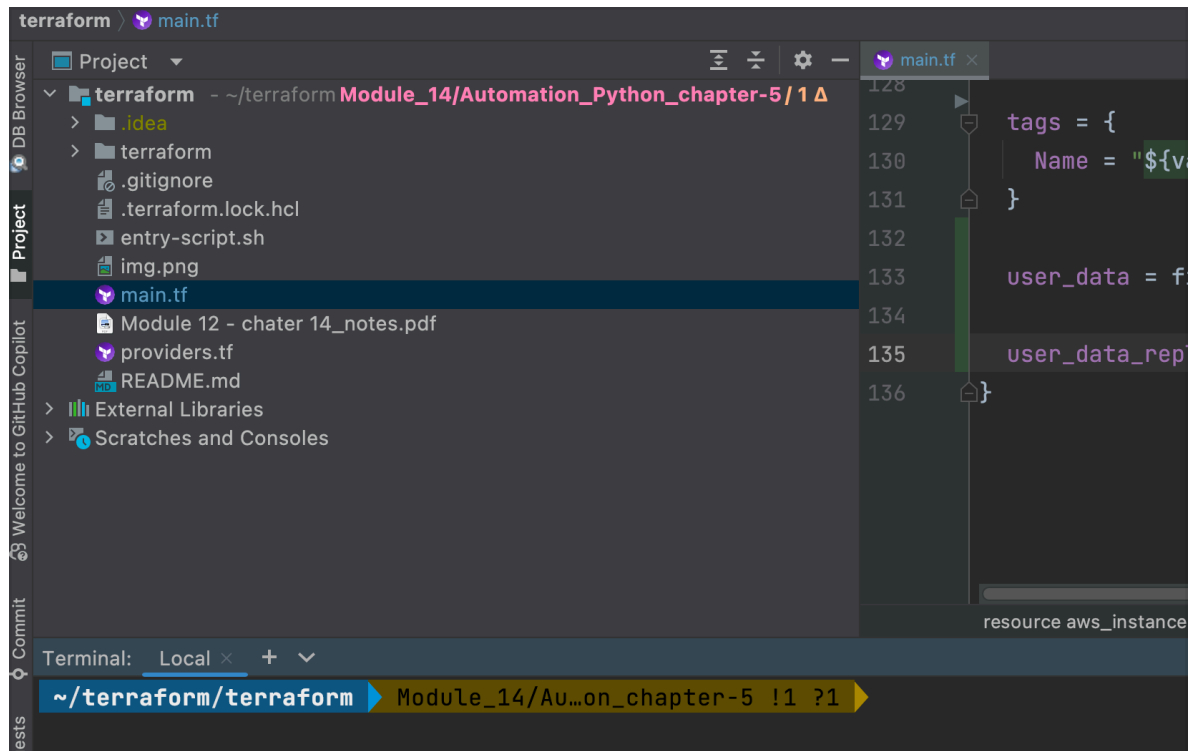
## Preparation:

Create EC2 instances with TF

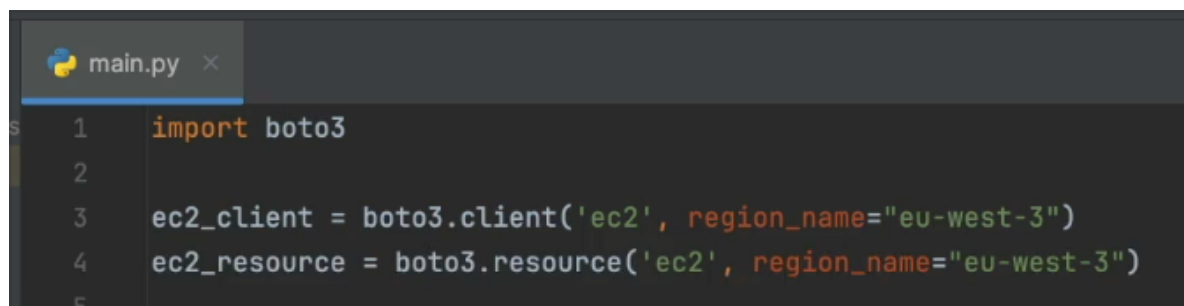
I created a branch from [https://github.com/AstridCaballero/Module\\_12\\_Terraform/blob/Module\\_12/Terraform\\_chapter-14/main.tf](https://github.com/AstridCaballero/Module_12_Terraform/blob/Module_12/Terraform_chapter-14/main.tf) and edit

the main.tf to be like Nana's DEMO

Branch -> Module\_14/Automation\_Python\_chapter-5



Now we edit the main.tf to create 3 servers, 3 EC2 instances in the region 'eu-west-3'



So we copy and paste the ec2 resource two times and rename the instances:

-> my app-server-two

-> myapp-server-three (line 158 and line 170)

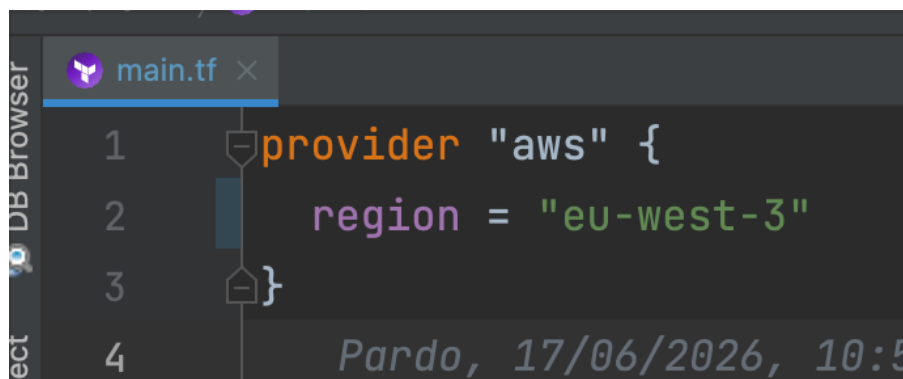
```

158 resource "aws_instance" "myapp-server-three" {
159     ami = data.aws_ssm_parameter.latest-amazon-linux-image.value
160     instance_type = var.instance_type
161
162     subnet_id = aws_subnet.myapp-subnet-1.id
163     vpc_security_group_ids = [aws_security_group.myapp-sg.id]
164     availability_zone = var.avail_zone
165
166     associate_public_ip_address = true
167     key_name = aws_key_pair.ssh-key.key_name
168
169     tags = {
170         Name = "${var.env_prefix}-server-three"
171     }
172
173     user_data = file("entry-script.sh")
174
175     user_data_replace_on_change = true
176 }

```

If need to create more servers there are ways in tf to do it without copy and paste.

Next, we edit the region

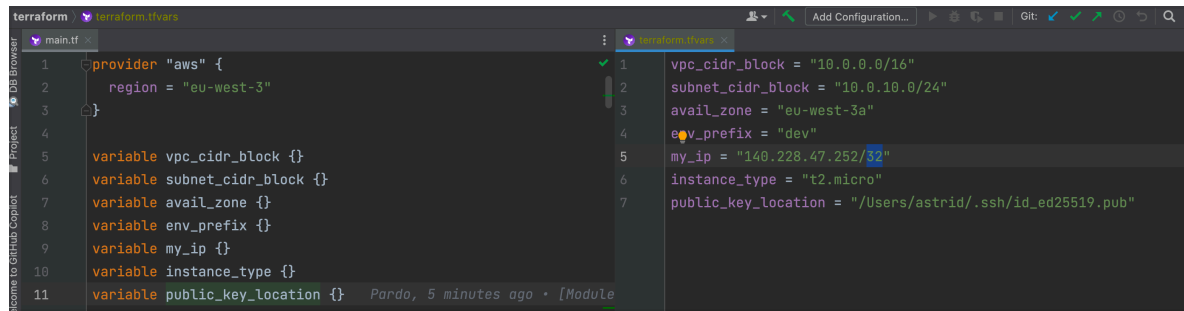


```

1 provider "aws" {
2     region = "eu-west-3"
3 }
4

```

And make sure to have the 'terraform.tfvars'



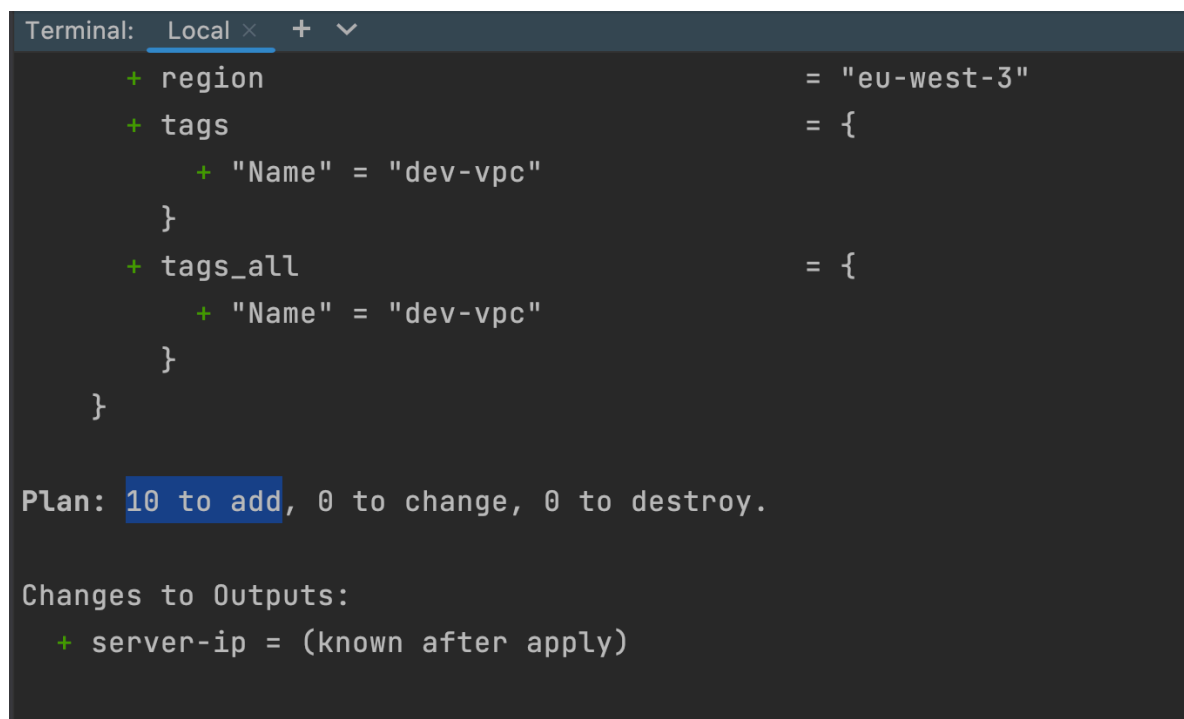
```
1 provider "aws" {
2   region = "eu-west-3"
3 }
4
5 variable vpc_cidr_block {}
6 variable subnet_cidr_block {}
7 variable avail_zone {}
8 variable env_prefix {}
9 variable my_ip {}
10 variable instance_type {}
11 variable public_key_location {}

1 vpc_cidr_block = "10.0.0.0/16"
2 subnet_cidr_block = "10.0.10.0/24"
3 avail_zone = "eu-west-3a"
4 env_prefix = "dev"
5 my_ip = "140.228.47.252/32"
6 instance_type = "t2.micro"
7 public_key_location = "/Users/astrid/.ssh/id_ed25519.pub"
```

Now we can check

-> terraform init

-> terraform plan



```
Terminal: Local x + v
+ region = "eu-west-3"
+ tags = {
  + "Name" = "dev-vpc"
}
+ tags_all = {
  + "Name" = "dev-vpc"
}
}

Plan: 10 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ server-ip = (known after apply)
```

-> terraform apply

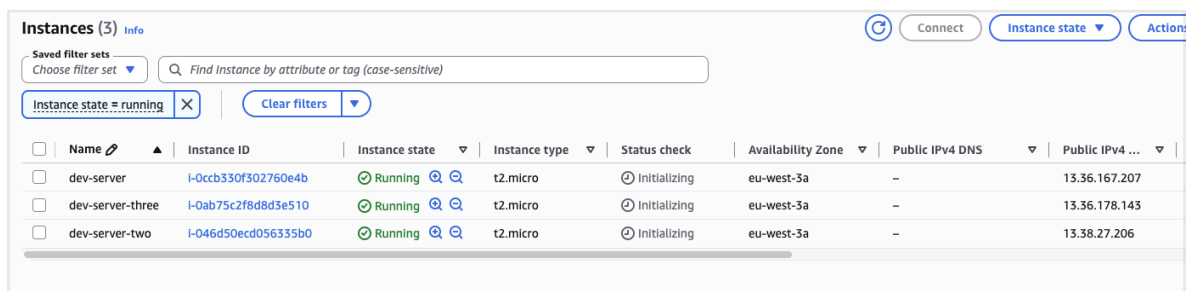
```
Terminal: Local x + v
aws_route_table_association.a-rtb-subnet: Creating...
aws_route_table_association.a-rtb-subnet: Creation complete after 0s [id=rtbassoc-08b18a6cdc19bd1a6]
aws_security_group.myapp-sg: Creation complete after 3s [id=sg-0b6ca848b0f386d5e]
aws_instance.myapp-server-two: Creating...
aws_instance.myapp-server-three: Creating...
aws_instance.myapp-server: Creating...
aws_instance.myapp-server-two: Still creating... [00m10s elapsed]
aws_instance.myapp-server-three: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 13s [id=i-0ccb330f302760e4b]
aws_instance.myapp-server-three: Creation complete after 13s [id=i-0ab75c2f8d8d3e510]
aws_instance.myapp-server-two: Creation complete after 13s [id=i-046d50ecd056335b0]

Apply complete! Resources: 10 added, 0 changed, 0 destroyed.

Outputs:

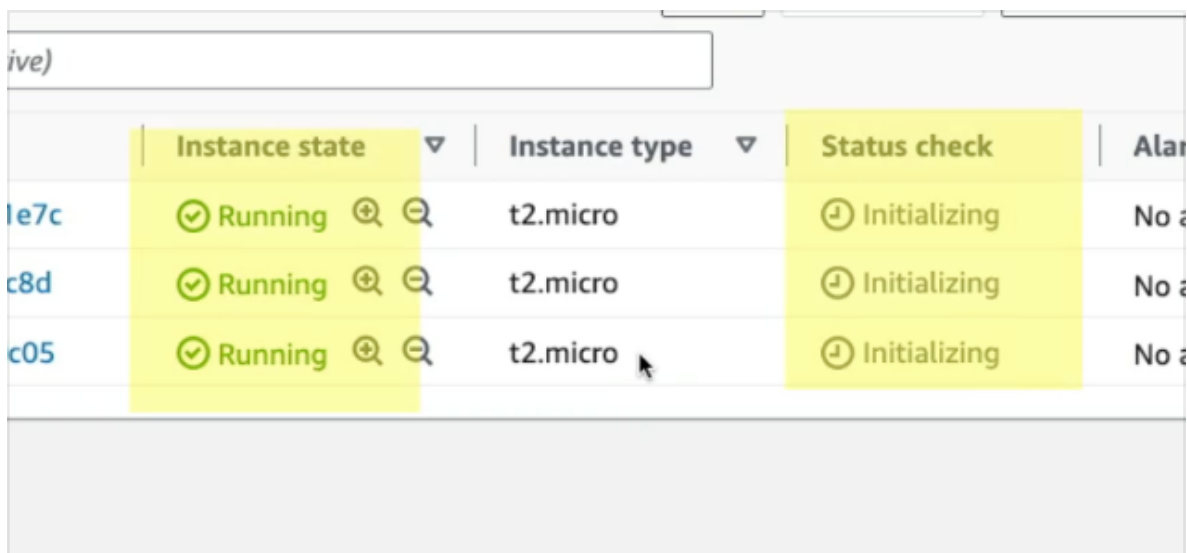
server-ip = "13.36.167.207"
```

And I have my 3 instances initialising



| Name             | Instance ID         | Instance state | Instance type | Status check | Availability Zone | Public IPv4 DNS | Public IPv4 ... |
|------------------|---------------------|----------------|---------------|--------------|-------------------|-----------------|-----------------|
| dev-server       | i-0ccb330f302760e4b | Running        | t2.micro      | Initializing | eu-west-3a        | -               | 13.36.167.207   |
| dev-server-three | i-0ab75c2f8d8d3e510 | Running        | t2.micro      | Initializing | eu-west-3a        | -               | 13.36.178.143   |
| dev-server-two   | i-046d50ecd056335b0 | Running        | t2.micro      | Initializing | eu-west-3a        | -               | 13.38.27.206    |

And this is the part we want to check with python -> in which state the instances are?



| Instance state | Instance type | Status check |
|----------------|---------------|--------------|
| Running        | t2.micro      | Initializing |
| Running        | t2.micro      | Initializing |
| Running        | t2.micro      | Initializing |

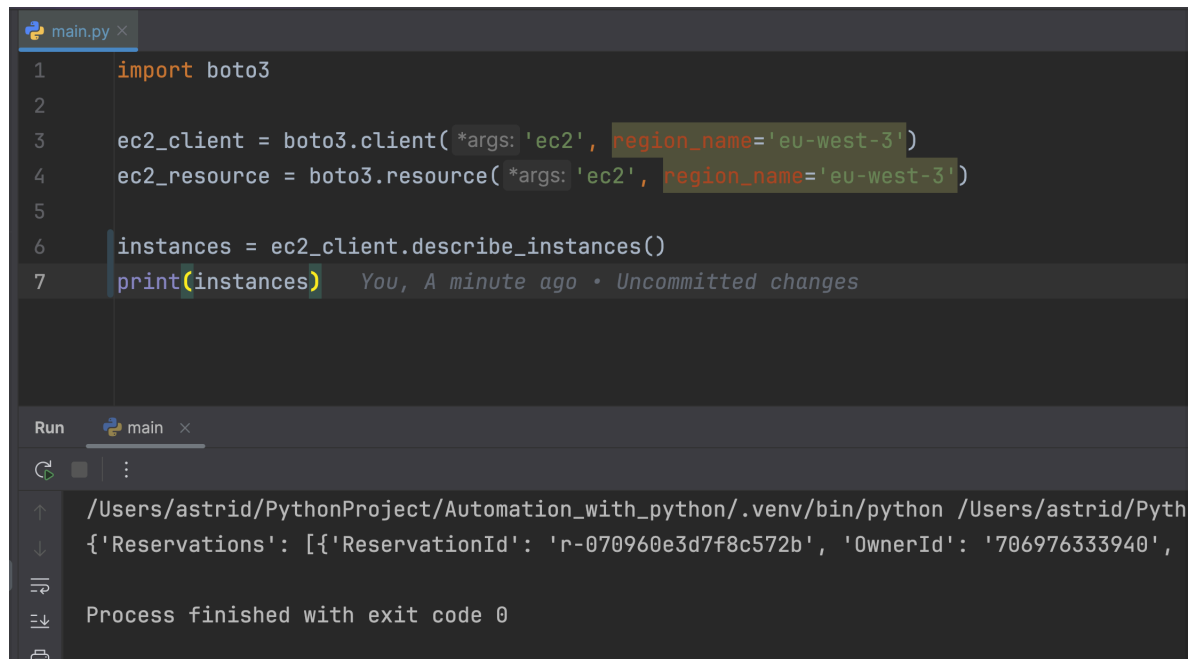
We want an overview of server status check because of the autoscaling (removing and adding servers constantly)

## Print EC2 Instance State

So now we go back to python

And we access the instances:

-> instances = ec2\_client.describe\_instances()



The screenshot shows a code editor with a file named 'main.py'. The code is as follows:

```
1 import boto3
2
3 ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-3')
4 ec2_resource = boto3.resource(*args: 'ec2', region_name='eu-west-3')
5
6 instances = ec2_client.describe_instances()
7 print(instances)
```

Below the code editor, there is a 'Run' button and a terminal window. The terminal shows the command being executed: `/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/Pyth`. The output is a JSON dictionary: `{'Reservations': [{'ReservationId': 'r-070960e3d7f8c572b', 'OwnerId': '706976333940',`. The terminal also shows 'Process finished with exit code 0'.

And we get a dictionary with a list of reservations.

But the response is not readable, so let's check the docs for function `.describe_instances()` [https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/client/describe\\_instances.html](https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/client/describe_instances.html)



RETURNS:

## Response Syntax

```
{
  'NextToken': 'string',
  'Reservations': [
    {
      'ReservationId': 'string',
      'OwnerId': 'string',
      'RequesterId': 'string',
      'Groups': [
        {
          'GroupId': 'string',
          'GroupName': 'string'
        }
      ],
      'Instances': [
        {
          'Architecture': 'i386'|'x86_64'|'arm64'|'x86_64_mac'|'arm64_mac'
          'BlockDeviceMappings': [
            {
              'DeviceName': 'string',
              'Ebs': {
                'AttachTime': datetime(2015, 1, 1),
                'DeleteOnTermination': True|False,
                'Status': 'attaching'|'attached'|'detaching'|'detach
                'VolumeId': 'string',
                'AssociatedResource': 'string',
                'VolumeOwnerId': 'string',
```

Q:

when I use `boto3 .describe_instances()` the response returns 'reservations' can you explain why 'reservations' keyword and not 'instances'?

A:

it's a historical artifact of EC2's API, not a boto3 design choice.

The reason: In the early EC2 API, a single `RunInstances` call was the unit of "reservation" — when you requested to launch instances, that request was called a Reservation, and it could contain multiple instances launched together in one call (e.g., asking for 5 instances at once). AWS tracked metadata at that reservation level: which security groups, which owner/requester ID, which reservation ID — separate from the instances themselves.

So the response structure reflects that: each Reservation is a wrapper containing:

- ReservationId
- OwnerId
- Groups (legacy EC2-Classical security groups)
- Instances — the actual list of instance dicts you usually care about

```
response = ec2.describe_instances()
for reservation in response['Reservations']:
    for instance in reservation['Instances']:
        print(instance['InstanceId'])
```

Why it's still there today: AWS almost never breaks backward compatibility

on API response shapes — once Reservations was the top-level key, it stayed that way even as the "reservation" concept became mostly vestigial for modern usage (VPC-based launches don't really expose reservations as a meaningful concept to users anymore). So you're left double-nesting through Reservations → Instances even though 95% of the time you just want a flat list of instances.

A common pattern to flatten it:

```
instances = [i for r in response['Reservations'] for i in r['Instances']]
```

Q:

so the return response is {  
 'NextToken': 'string',  
 'Reservations': [

and if I want to get the list of reservation I have to write:  
response['Reservations'] why I have to provide the key 'Reservations'?

A:

That's just how Python dictionaries work, not something specific to boto3 or AWS.

describe\_instances() returns a regular Python dict. A dict doesn't store values in a fixed order you can grab "the first thing" from by default — each value is stored under a key, and you retrieve it by naming that key:

```
response = {  
    'NextToken': 'string',  
    'Reservations': [ ... ],  
    'ResponseMetadata': { ... }  
}
```

This dict has multiple top-level entries — NextToken, Reservations, ResponseMetadata, etc. If you just wrote response, you'd get the whole thing (all three). To get only the reservations list, you tell Python which value you want by its key:

```
response['Reservations'] # → the list  
response['NextToken']    # → the pagination token, if present
```

It's the same reason you'd write person['name'] instead of just person to get someone's name out of {'name': 'Astrid', 'age': 30} — the key is the label that says which piece of the dict you're asking for.

If it were a list instead of a dict, you'd index by position (response[0]), but boto3 always returns a dict here because the response bundles multiple named fields together (the reservations and metadata like NextToken), not just one

array.

So instances are grouped into reservations. we want to get to our instances and from that we want the 'State'

```
    ],
    'InstanceId': 'string',
    'ImageId': 'string',
    'State': {
      'Code': 123,
      'Name': 'pending'|'running'|'shutting-down'|'terminated'|'st
    },
    'PrivateDnsName': 'string',
```

This 'state' is the 'instance state' column

| Instance state ▾ |
|------------------|
| ✓ Running 🔍 🔍    |
| ✓ Running 🔍 🔍    |
| ✓ Running 🔍 🔍    |
|                  |
|                  |

Actually we want to access state-> Name for each instance

Let's get the reservations:

```
-> reservations = ec2_client.describe_instances()
-> for reservation in reservations['Reservations']:
->   print(reservation['Instances'])
```

```
main.py x
1 import boto3
2
3 ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-3')
4 ec2_resource = boto3.resource(*args: 'ec2', region_name='eu-west-3')
5
6 reservations = ec2_client.describe_instances() You, Moments ago • Uncommitted cha
7 for reservation in reservations['Reservations']:
8     print(reservation['Instances'])

Run main x
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/Pyth
[{'Architecture': 'x86_64', 'BlockDeviceMappings': [{'DeviceName': '/dev/xvda', 'Ebs':
[{'Architecture': 'x86_64', 'BlockDeviceMappings': [{'DeviceName': '/dev/xvda', 'Ebs':
[{'Architecture': 'x86_64', 'BlockDeviceMappings': [{'DeviceName': '/dev/xvda', 'Ebs':
Process finished with exit code 0
```

We get 3 reservations. And each reservation has 1 instance.

Below is the structure of the response from the documentation

```
main.py response.py x
1 {
2     'NextToken': 'string',
3     'Reservations': [
4         {
5             'ReservationId': 'string',
6             'OwnerId': 'string', You, Moments ago
7             'RequesterId': 'string',
8             'Groups': [...],
14            'Instances': [...]
293        },
294    ],
295 }
```

Q:

I have 3 instances and the response is 3 reservations each with an instance, why not one reservation with the 3 instances?

A:

Because that depends on how the instances were launched, not how many

exist.

Each RunInstances API call creates exactly one Reservation, and that reservation contains however many instances you asked for in that single call. So:

- If you ran `run_instances(..., MinCount=3, MaxCount=3)` once, you'd get 1 reservation with 3 instances inside it.
- If you launched them separately — 3 different calls (e.g., three separate clicks in the console, three separate `run_instances` calls, three separate Auto Scaling actions, or on three different days) — you get 3 reservations, each with 1 instance.

`describe_instances()` isn't grouping instances by anything meaningful today (like "these 3 are related") — it's just showing you the historical launch-batch record for each instance. Since your 3 instances were each created in their own launch event, each one sits in its own reservation.

This is exactly why the reservation concept feels like leftover baggage — it reflects launch history, not any current relationship between the instances (they could be totally unrelated). That's why almost everyone just flattens it:

```
instances = [i for r in response['Reservations'] for i in r['Instances']]
```

Q:

i created the instances using terraform:



```
119
120 +resource "aws_instance" "myapp-server" {...}
139
140 +resource "aws_instance" "myapp-server-two" {...}
159
160 +resource "aws_instance" "myapp-server-three" {...}
```

so by repeating the code 3 times I made 3 calls?

A:

Exactly right. Each `aws_instance` resource block is a separate resource in Terraform's graph, and under the hood the AWS provider makes one RunInstances API call per resource — so your 3 blocks (`myapp-server`, `myapp-server-two`, `myapp-server-three`) triggered 3 separate RunInstances calls, each creating its own 1-instance reservation. That's why `describe_instances()` shows 3 reservations with 1 instance each.

If you'd used `count` or `for_each` on a single resource block instead:

```
resource "aws_instance" "myapp-server" {
  count = 3
  ami    = data.aws_ssm_parameter.latest-amazon-linux-image.value
  instance_type = var.instance_type
  ...
  tags = {
    Name = "${var.env_prefix}-server-${count.index}"
  }
}
```

Terraform still calls `RunInstances` once per instance internally (the AWS API doesn't have a native "create N instances with different tags" call when tags differ per instance anyway), so you'd still likely see 3 reservations — the reservation grouping isn't really something Terraform controls either way. It only collapses to one reservation if a single `RunInstances` call requests multiple identical instances at once (same tags, same everything), which isn't how the AWS provider issues the request regardless of `count/for_each` vs. copy-pasted blocks.

`Instances` is a list of dictionaries, each dictionary is an instance and each instance dictionary has these key-pairs:

```
14      'Instances': [
15        {
16          'Architecture': 'i386'|'x86_64'|'arm64'|'x86_64',
17        >   'BlockDeviceMappings': [...],
36          'ClientToken': 'string',
37          'EbsOptimized': True|False,
38          'EnaSupport': True|False,
39          'Hypervisor': 'ovm'|'xen',
40        >   'IamInstanceProfile': {...},
44          'InstanceLifecycle': 'spot'|'scheduled'|'on-demand',
45        >   'ElasticGpuAssociations': [...],
53        >   'ElasticInferenceAcceleratorAssociations': [...],
61        >   'NetworkInterfaces': [...],
144          'OutpostArn': 'string',
145          'RootDeviceName': 'string',
146          'RootDeviceType': 'ebs'|'instance-store',
```

...

```
main.py response.py x
217 > 'NetworkPerformanceOptions': {...},
220 > 'Operator': {...},
225 > 'SecondaryInterfaces': [...],
250 'InstanceId': 'string',
251 'ImageId': 'string',
252 ✦ 'State': { You, 14 minutes ago • Uncommitted changes
253     'Code': 123,
254     'Name': 'pending' | 'running' | 'shutting-down' | 'terminated' |
255 },
256 'PrivateDnsName': 'string',
```

And one of them is 'State' see line 252 🙋 and 'State' is a dictionary ( 'State' is the key and the {} dictionary is the value)

So our code to get the dictionary of each instance:

```
5
6 reservations = ec2_client.describe_instances()
7 for reservation in reservations['Reservations']:
8     instances = reservation['Instances']
9     for instance in instances:
10         💡 print(instance) You, Moments ago • Uncommitted
```

And now we can access the values of the key 'State'  
-> instance['State']

```
main.py × response.py
1 import boto3
2
3 ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-3')
4 ec2_resource = boto3.resource(*args: 'ec2', region_name='eu-west-3')
5
6 reservations = ec2_client.describe_instances()
7 for reservation in reservations['Reservations']:
8     instances = reservation['Instances']
9     for instance in instances:
10         print(instance['State']) You, Moments ago • Uncommitted changes

Run main ×
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonPr
{'Code': 16, 'Name': 'running'}
{'Code': 16, 'Name': 'running'}
{'Code': 16, 'Name': 'running'}
Process finished with exit code 0
```

We get a 'State' dictionary per instance

We don't need the 'Code' only the 'Name' so from each 'State' dictionary we get the 'Name'

-> instance['State']['Name']

```
main.py × response.py
1 import boto3
2
3 ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-3')
4 ec2_resource = boto3.resource(*args: 'ec2', region_name='eu-west-3')
5
6 reservations = ec2_client.describe_instances()
7 for reservation in reservations['Reservations']:
8     instances = reservation['Instances']
9     for instance in instances:
10         print(instance['State']['Name']) You, Moments ago • Uncommitted char

Run main ×
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid
running
running
running
Process finished with exit code 0
```

Each instance also has a key 'InstanceId'



```

225     'SecondaryInterfaces': [...],
250     'InstanceId': 'string',    You, Today
251     'ImageId': 'string',
252     'State': {
253         'Code': 123,
254         'Name': 'pending' | 'running' | 'shut
255     },

```

So we can print a message using the value of 'InstanceId'

-> `print(f"Instance {instance['InstanceId']} is {instance['State']['Name']}")`

The screenshot shows a code editor with two files: `main.py` and `response.py`. The `main.py` file contains the following Python code:

```

1  import boto3
2
3  ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-3')
4  ec2_resource = boto3.resource(*args: 'ec2', region_name='eu-west-3')
5
6  reservations = ec2_client.describe_instances()
7  for reservation in reservations['Reservations']:
8      instances = reservation['Instances']
9      for instance in instances:
10         print(f"Instance {instance['InstanceId']} is {instance['State']['Name']}")

```

Below the code editor is a terminal window titled "Run" with a sub-tab "main". It shows the output of the script:

```

/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/Pytho
Instance i-0ab75c2f8d8d3e510 is running
Instance i-0ccb330f302760e4b is running
Instance i-046d50ecd056335b0 is running
Process finished with exit code 0

```

So If I now go to Terraform and remove one of the EC2 instances to have only 2 instances in total:

```
terraform > main.tf
main.tf x
108 resource "aws_route_table" "myapp-route-table-1a" {
109 }
110
111 resource "aws_key_pair" "ssh-key" {
112     key_name     = "myapp-key"
113     public_key = file(var.public_key_location)
114 }
115
116 output "server-ip" {
117     value = aws_instance.myapp-server.public_ip
118 }
119
120 resource "aws_instance" "myapp-server" {...}
139
140 resource "aws_instance" "myapp-server-two" {...} | Pardo, Tod
159
```

-> terraform plan

```
Plan: 0 to add, 0 to change, 1 to destroy.
```

-> terraform apply

```
Apply complete! Resources: 0 added, 0 changed, 1 destroyed.
```

```
Outputs:
```

```
server-ip = "13.36.167.207"
```

```
~/terraform Module_14/Au...on_chapter-5 !1 ?1
```

| Instances (3) Info                                 |                  |                     |                |               |                   |              |                   |                |
|----------------------------------------------------|------------------|---------------------|----------------|---------------|-------------------|--------------|-------------------|----------------|
| Find Instance by attribute or tag (case-sensitive) |                  |                     |                |               |                   |              |                   |                |
| <input type="checkbox"/>                           | Name             | Instance ID         | Instance state | Instance type | Status check      | Alarm status | Availability Zone | Public IPv4 DN |
| <input type="checkbox"/>                           | dev-server-two   | i-0b2327a695d511e7c | Running        | t2.micro      | 2/2 checks passed | No alarms    | eu-west-3a        | -              |
| <input type="checkbox"/>                           | dev-server-three | i-0cc03054c5922ac8d | Shutting-d...  | t2.micro      | -                 | No alarms    | eu-west-3a        | -              |
| <input type="checkbox"/>                           | dev-server       | i-06dcd876ce1050c05 | Running        | t2.micro      | 2/2 checks passed | No alarms    | eu-west-3a        | -              |

And if we run the python script:

```

Run main x
Instance i-0b2327a695d511e7c is running
Instance i-0cc03054c5922ac8d is shutting-down
Instance i-06dcd876ce1050c05 is running
Process finished with exit code 0

```

I forgot to removed the filter of the instances in my AWS UI so I didn't get the 'shutting down' instance state in a screenshot

| Instances (2) Info                                 |                |                     |                |               |                   |                   |                 |                 |            |
|----------------------------------------------------|----------------|---------------------|----------------|---------------|-------------------|-------------------|-----------------|-----------------|------------|
| Find Instance by attribute or tag (case-sensitive) |                |                     |                |               |                   |                   |                 |                 |            |
| Instance state = running                           |                |                     |                |               |                   |                   |                 |                 |            |
| <input type="checkbox"/>                           | Name           | Instance ID         | Instance state | Instance type | Status check      | Availability Zone | Public IPv4 DNS | Public IPv4 ... | Elastic IP |
| <input type="checkbox"/>                           | dev-server     | i-0ccb330f302760e4b | Running        | t2.micro      | 2/2 checks passed | eu-west-3a        | -               | 13.36.167.207   | -          |
| <input type="checkbox"/>                           | dev-server-two | i-046d50ecd056335b0 | Running        | t2.micro      | 2/2 checks passed | eu-west-3a        | -               | 13.38.27.206    | -          |

When I realised about the filter and I removed it, the instance state was already 'terminated'

| Instances (3) Info                                 |                  |                     |                |               |                   |                   |                 |                 |            |
|----------------------------------------------------|------------------|---------------------|----------------|---------------|-------------------|-------------------|-----------------|-----------------|------------|
| Find Instance by attribute or tag (case-sensitive) |                  |                     |                |               |                   |                   |                 |                 |            |
| <input type="checkbox"/>                           | Name             | Instance ID         | Instance state | Instance type | Status check      | Availability Zone | Public IPv4 DNS | Public IPv4 ... | Elastic IP |
| <input type="checkbox"/>                           | dev-server       | i-0ccb330f302760e4b | Running        | t2.micro      | 2/2 checks passed | eu-west-3a        | -               | 13.36.167.207   | -          |
| <input type="checkbox"/>                           | dev-server-three | i-0ab75c2f8d8d3e510 | Terminated     | t2.micro      | -                 | eu-west-3a        | -               | -               | -          |
| <input type="checkbox"/>                           | dev-server-two   | i-046d50ecd056335b0 | Running        | t2.micro      | 2/2 checks passed | eu-west-3a        | -               | 13.38.27.206    | -          |

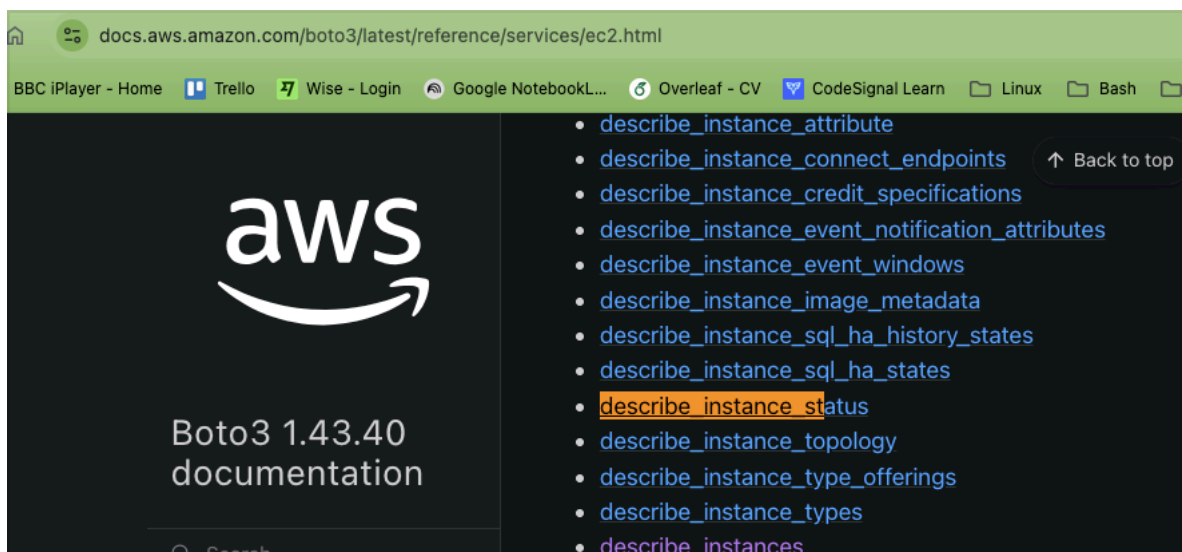
When I run my python script I get this

```
Run main x
/Users/astrid/PythonProject/Automation_with_python/.venv
Instance i-0ab75c2f8d8d3e510 is terminated
Instance i-0ccb330f302760e4b is running
Instance i-046d50ecd056335b0 is running
Process finished with exit code 0
```

## Print EC2 Status check

To get the 'status' data we need to use another function ->

`describe_instance_status()` [https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/client/describe\\_instance\\_status.html](https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/client/describe_instance_status.html)



We call this function on an EC2 **client**

EC2 / Client / describe\_instance\_status

# describe\_instance\_status

EC2.Client.describe\_instance\_status(\*\*kwargs)

Describes the status of the specified instances or all of your instances.

And this is the response we get back

## Response Syntax

[↑ Back to top](#)

```
{
  'InstanceStatuses': [
    {
      'AvailabilityZone': 'string',
      'AvailabilityZoneId': 'string',
      'OutpostArn': 'string',
      'Operator': {
        'Managed': True|False,
        'Principal': 'string',
        'HiddenByDefault': True|False
      },
      'Events': [
        {
          'InstanceEventId': 'string',
          'Code': 'instance-reboot'|'system-reboot'|'system-maintenance'|
          'Description': 'string',
          'NotAfter': datetime(2015, 1, 1),
          'NotBefore': datetime(2015, 1, 1),
          'NotBeforeDeadline': datetime(2015, 1, 1)
        },
      ],
      'InstanceId': 'string',
      'InstanceState': {
        'Code': 123,
        'Name': 'pending'|'running'|'shutting-down'|'terminated'|'stopping'|
      },
      'InstanceStatus': {
        'Details': [
          {
            'ImpairedSince': datetime(2015, 1, 1),
            'Name': 'reachability',
            'Status': 'passed'|'failed'|'insufficient-data'|'initializing'
          },
        ],
        'Status': 'ok'|'impaired'|'insufficient-data'|'not-applicable'|'initializing'
      },
      'SystemStatus': {
        'Details': [
          {
            'ImpairedSince': datetime(2015, 1, 1),
            'Name': 'reachability',
            'Status': 'passed'|'failed'|'insufficient-data'|'initializing'
          },
        ],
        'Status': 'ok'|'impaired'|'insufficient-data'|'not-applicable'|'initializing'
      }
    },
  ],
}
```

And for the 'status check' UI column

|                     |
|---------------------|
| Status check        |
| ✔ 2/2 checks passed |
| -                   |
| ✔ 2/2 checks passed |
|                     |
|                     |

we need 'InstanceStatus' and 'SystemStatus'

Here we can see better the response structure

```
main.py response.py InstanceStatusResponse.py ×
1 {
2     'InstanceStatuses': [
3         > { },
58     ],
59     ⚡ 'NextToken': 'string'
60 }
```

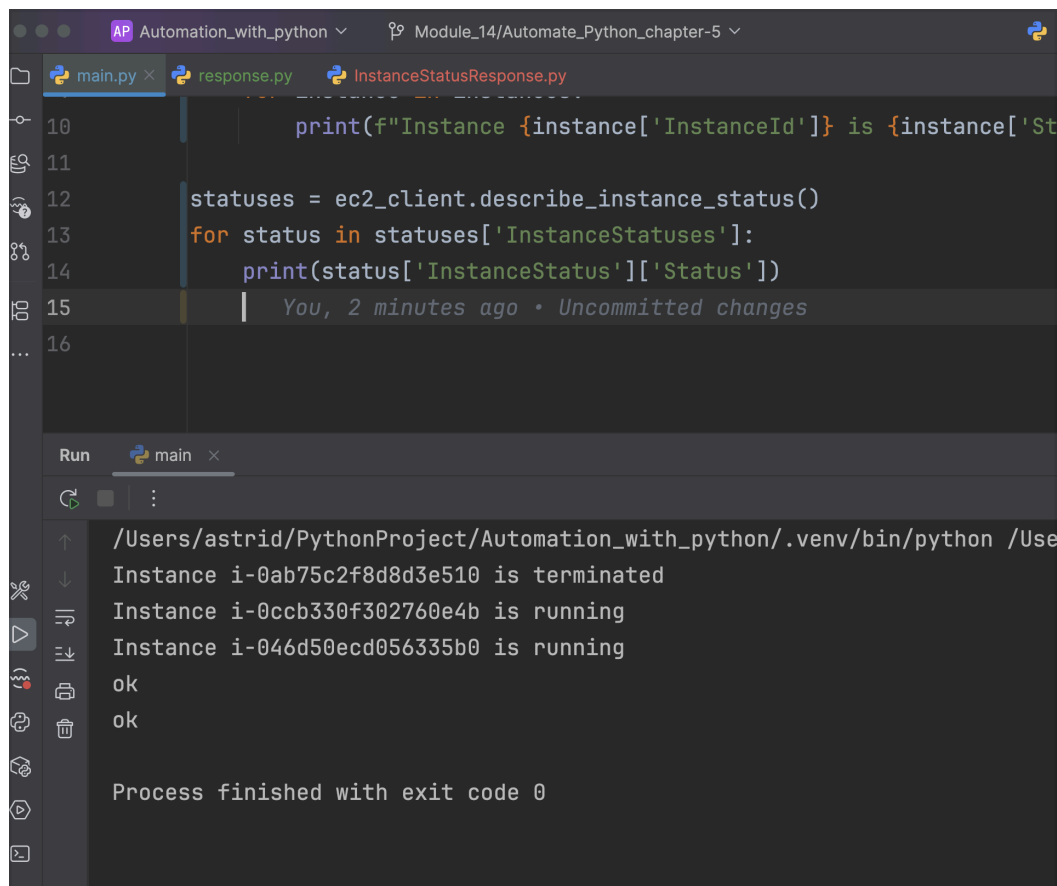
```
main.py response.py InstanceStatusResponse.py x
1  {
2      'InstanceStatuses': [
3          {
4              'AvailabilityZone': 'string',
5              'AvailabilityZoneId': 'string',
6              'OutpostArn': 'string',
7              'Operator': {...},
12             'Events': [...],
22             'InstanceId': 'string',
23             'InstanceState': {...},
27             'InstanceStatus': {...},
37             'SystemStatus': {...},
47             'AttachedEbsStatus': {...}
57         },
58     ],
59     'NextToken': 'string'
60 }
```

Now let's get the 'InstanceStatus' of each dictionary of the list 'InstanceStatuses'

```
27     'InstanceStatus': {
28         'Details': [
29             {
30                 'ImpairedSince': datetime(2015, 1, 1),
31                 'Name': 'reachability',
32                 'Status': 'passed' | 'failed' | 'insufficient-data' | 'initializing'
33             },
34         ],
35         'Status': 'ok' | 'impaired' | 'insufficient-data' | 'not-applicable' | 'initializing'
36     },
```

And from that we want 'Status'

And here is the code:



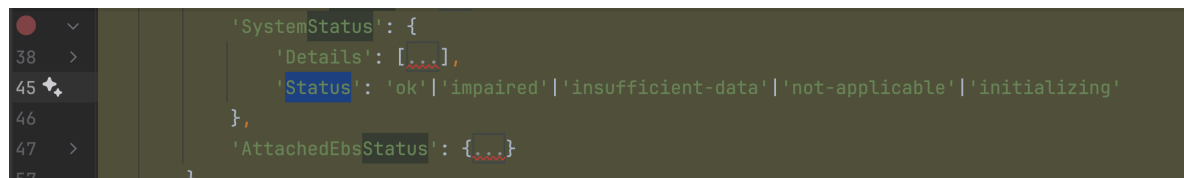
```
10         print(f"Instance {instance['InstanceId']} is {instance['St
11
12     statuses = ec2_client.describe_instance_status()
13     for status in statuses['InstanceStatuses']:
14         print(status['InstanceStatus']['Status'])
15         | You, 2 minutes ago • Uncommitted changes
16
```

Run main

```
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Use
Instance i-0ab75c2f8d8d3e510 is terminated
Instance i-0ccb330f302760e4b is running
Instance i-046d50ecd056335b0 is running
ok
ok

Process finished with exit code 0
```

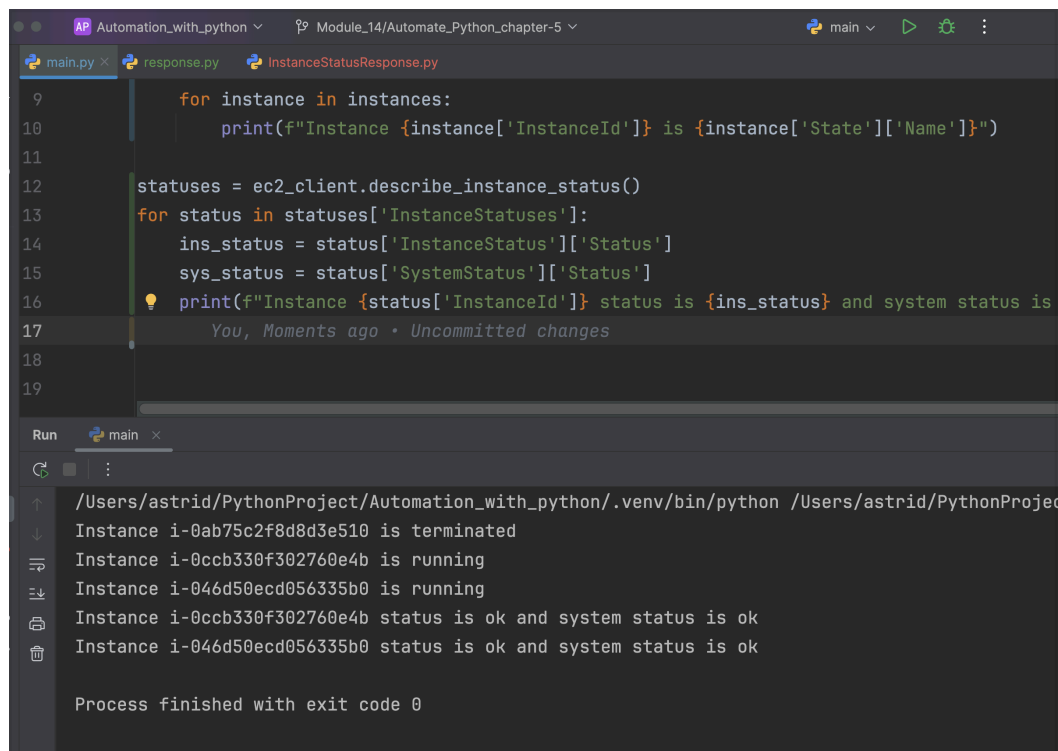
Now let's get 'Status' from 'SystemStatus'



```
38     'SystemStatus': {
45         'Details': [...],
46         'Status': 'ok' | 'impaired' | 'insufficient-data' | 'not-applicable' | 'initializing'
47     },
57     'AttachedEbsStatus': {...}
```

And here is the code





```
9         for instance in instances:
10             print(f"Instance {instance['InstanceId']} is {instance['State']['Name']}")
11
12     statuses = ec2_client.describe_instance_status()
13     for status in statuses['InstanceStatuses']:
14         ins_status = status['InstanceState']['Status']
15         sys_status = status['SystemStatus']['Status']
16         print(f"Instance {status['InstanceId']} status is {ins_status} and system status is {sys_status}")
17
18
19
```

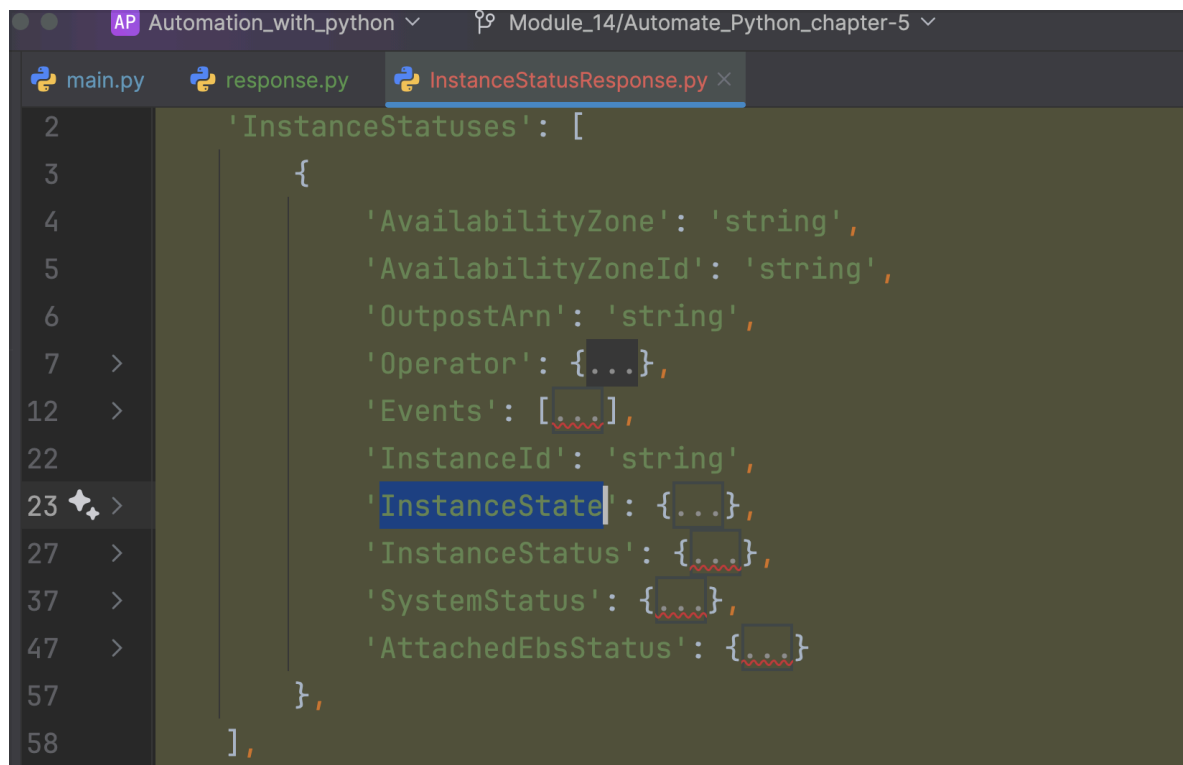
Run main

```
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automation_with_python/main.py
Instance i-0ab75c2f8d8d3e510 is terminated
Instance i-0ccb330f302760e4b is running
Instance i-046d50ecd056335b0 is running
Instance i-0ccb330f302760e4b status is ok and system status is ok
Instance i-046d50ecd056335b0 status is ok and system status is ok

Process finished with exit code 0
```

## Get everything in 1 AWS API Call

Choking the response of function `describe_instance_status()` it also includes the 'InstanceState' so we can actually use only one function to get the state and the status!



```
2     'InstanceStatuses': [
3         {
4             'AvailabilityZone': 'string',
5             'AvailabilityZoneId': 'string',
6             'OutpostArn': 'string',
7             'Operator': {...},
12            'Events': [...],
22            'InstanceId': 'string',
23            'InstanceState': {...},
27            'InstanceStatus': {...},
37            'SystemStatus': {...},
47            'AttachedEbsStatus': {...}
57        },
58    ],
```

```

23      'InstanceState': {
24          'Code': 123,
25          'Name': 'pending' | 'running'
26      },

```

And this is the code:

```

Automation_with_python Module_14/Automate_Python_chapter-5
main.py response.py InstanceStatusResponse.py
5
6     # reservations = ec2_client.describe_instances()
7     # for reservation in reservations['Reservations']:
8     #     instances = reservation['Instances']
9     #     for instance in instances:
10    #         print(f"Instance {instance['InstanceId']} is {instance['State']['Name']}")
11
12    statuses = ec2_client.describe_instance_status()
13    for status in statuses['InstanceStatuses']:
14        ins_status = status['InstanceState']['Status']
15        sys_status = status['SystemStatus']['Status']
16        state = status['InstanceState']['Name']

```

And we clean it a bit

```

Automation_with_python Module_14/Automate_Python_chapter-5
main.py response.py InstanceStatusResponse.py
1    import boto3
2
3    ec2_client = boto3.client(*args: 'ec2', region_name='eu-west-3')
4    ec2_resource = boto3.resource(*args: 'ec2', region_name='eu-west-3')
5
6    statuses = ec2_client.describe_instance_status()
7    for status in statuses['InstanceStatuses']:
8        ins_status = status['InstanceState']['Status']
9        sys_status = status['SystemStatus']['Status']
10       state = status['InstanceState']['Name']
11       print(f"Instance {status['InstanceId']} is {state} with instance status {ins_status} and system
12
Run main
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automation
Instance i-0ccb330f302760e4b is running with instance status ok and system status ok
Instance i-046d50ecd056335b0 is running with instance status ok and system status ok
Process finished with exit code 0

```